

中山大學

SUN YAT-SEN UNIVERSITY

实验 0x02 实验报告

中断处理

姓　　名：　何亮

学　　号：　24325005

教学班号：　202520804（陈鹏飞老师班级）

专　　业：　计算机科学与技术

院　　系：　计算机学院

2025 学年第二学期

1. GDT 与 TSS

参考给定的特权栈声明代码，加入 `page_fault` 和 `double_fault` 的栈的声明，如下：

```
// FIXED: fill tss.interrupt_stack_table with the static stack buffers like above
// You can use `tss.interrupt_stack_table[DOUBLE_FAULT_IST_INDEX as usize]`
        tss.interrupt_stack_table[DOUBLE_FAULT_IST_INDEX as usize] = {
            const STACK_SIZE: usize = IST_SIZES[DOUBLE_FAULT_IST_INDEX as usize];
            static mut STACK: [u8; STACK_SIZE] = [0; STACK_SIZE];
            let stack_start = VirtAddr::from_ptr(addr_of_mut!(STACK));
            let stack_end = stack_start + STACK_SIZE as u64;
            info!(
                "Double Fault IST : 0x{:016x}-0x{:016x}",
                stack_start.as_u64(),
                stack_end.as_u64()
            );
            stack_end
        };
        tss.interrupt_stack_table[PAGE_FAULT_IST_INDEX as usize] = {
            const STACK_SIZE: usize = IST_SIZES[PAGE_FAULT_IST_INDEX as usize];
            static mut STACK: [u8; STACK_SIZE] = [0; STACK_SIZE];
            let stack_start = VirtAddr::from_ptr(addr_of_mut!(STACK));
            let stack_end = stack_start + STACK_SIZE as u64;
            info!(
                "Page Fault IST   : 0x{:016x}-0x{:016x}",
                stack_start.as_u64(),
                stack_end.as_u64()
            );
            stack_end
        };
```

2. 注册中断处理程序

在 `interrupt/mod.rs` 中调用对应的方法初始化 APIC，并进行异常处理

```
// FIXED: check and init APIC
if XApic::support() {
    let mut lapic = unsafe {
        XApic::new(physical_to_virtual(LAPIC_ADDR))
    };
    lapic.cpu_init();
} else {
    panic!("xAPIC is not supported!");
}
```

在 `exception.rs` 中参考给出的代码为各种 CPU 异常注册异常处理程序，限于篇幅，具体请见 `interrupt/exception.rs`。

3. 初始化 APIC

- 检测系统中是否存在 APIC

```
// FIXED: Check CPUID to see if xAPIC is supported.
CpuId::new().get_feature_info().map(
    |f| f.has_apic()
).unwrap_or(false)
```

- 启用 APIC 并设置 spurious interrupt vector

具体而言就是把第 `8` 位的使能 `EN` 位设为 `1`，高位不变，同时设置第 `4~7` 位的 `Vector` 设为 `Irq::Spurious`

```
// FIXED: Enable local APIC; set spurious interrupt vector.
let mut spiv = self.read(0xF0);
spiv |= 1<<8; // set EN bit
// clear and set Vector
spiv &= !(0xFF);
piv |= Interrupts::IrqBase as u32 + Irq::Spurious as u32;
self.write(0xF0, spiv);
```

- 设置 LVT 寄存器

需要将 Timer 的 `Vector` 设置为 `Irq::Timer`，并将 `M` 设置为 `0` 表示开中断，而后设置 `TP` 为 `1`，表示定时器周期模式

```
let mut lvt_timer = self.read(0x320);
// clear and set Vector
lvt_timer &= !(0xFF);
lvt_timer |= Interrupts::IrqBase as u32 + Irq::Timer as u32;
lvt_timer &= !(1 << 16); // clear Mask
lvt_timer |= 1 << 17; // set Timer Periodic Mode
self.write(0x320, lvt_timer);
```

- 禁用 LINT0 和 LINT1 以及 PCINT

将中断使能位设置为 `1` 表示关中断即可

```
// FIXED: Disable logical interrupt lines (LINT0, LINT1)
self.write(0x350, 1 << 16); // set Mask LINT0
self.write(0x360, 1 << 16); // set Mask LINT1
// FIXED: Disable performance counter overflow interrupts (PCINT)
self.write(0x340, 1 << 16); // set Mask PCINT
```

- 设置计时器的分频系数和计数周期

这里设置为频率设置为 `1`，周期设置为 `0x20000`

```
self.write(0x3E0, 0b1011); // set Timer Divide to 1
// self.write(0x3E0, 0b0000); // Test: set Timer Divide to 2
// self.write(0x3E0, 0b1000); // Test set Timer Divide to 4
self.write(0x380, 0x20000); // set initial count to 0x20000
```

- 设置错误中断 LVT Error 到对应的中断向量号

```
// FIXED: Map error interrupt to IRQ_ERROR.
self.write(0x370, Interrupts::IrqBase as u32 + Irq::Error as u32);
```

- 连续写入两次 0 以清除错误状态寄存器

```
// FIXED: Clear error status register (requires back-to-back
// writes).
self.write(0x280, 0); // clear error status register
self.write(0x280, 0); // clear error status register Again
```

- 向 EOI 寄存器写入 0 以确认任何挂起的中断

```
// FIXED: Ack any outstanding interrupts.
self.write(0x0b0, 0); // write EOI as zero
```

- 设置 ICR 寄存器

按照实验文档进行配置即可

```
// FIXED: Send an Init Level De-Assert to synchronise arbitration
// ID's.
self.write(0x310, 0); // set ICR 0x310
const BCAST: u32 = 1 << 19;
const INIT: u32 = 5 << 8;
const TMLV: u32 = 1 << 15; // TM = 1, LV = 0
self.write(0x300, BCAST | INIT | TMLV); // set ICR 0x300
const DS: u32 = 1 << 12;
while self.read(0x300) & DS != 0 {} // wait for delivery status
```

- 设置 TPR 寄存器为 0，允许接收中断

```
// FIXED: Enable interrupts on the APIC (but not on the processor).
self.write(0x080, 0); // clear TPR to enable all interrupts
```

4. 时钟中断

根据实验 0x00 的思考题，使用 `static mut` 是 unsafe 的，因此我们设置时钟的时候需要使用 `AtomicU64` 来保证我们每次对时钟的操作都是原子操作，即使是在多线程下仍然是安全的

```
static COUNTER: AtomicU64 = AtomicU64::new(0);

#[inline]
pub fn read_counter() -> u64 {
    // FIXED: load counter value
    COUNTER.load(core::sync::atomic::Ordering::SeqCst)
}

#[inline]
pub fn inc_counter() -> u64 {
    // FIXED: read counter value and increase it
    COUNTER.fetch_add(1, core::sync::atomic::Ordering::SeqCst) + 1 as u64
}
```

我们此时尝试使用 `make run` 运行，如下图

```
linkzelda@LAPTOP-QFAEHS2I:~/OS-lab$ make run
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel::interrupt::clock - Tick! @65536
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel::interrupt::clock - Tick! @131072
```

约每隔 12 条 `INFO` 时钟会 Tick! 一次，我们可以通过更改 TDCR 的值为 `0b0000` 让时钟的频率中断的频率减半，更改后的效果如下

```
// self.write(0x3E0, 0b1011); // set Timer Divide to 1
self.write(0x3E0, 0b0000); // Test: set Timer Divide to 2
// self.write(0x3E0, 0b1000); // Test set Timer Divide to 4
self.write(0x380, 0x20000); // set initial count to 0x20000
```

```
linkzelda@LAPTOP-QFAEHS2I:~/OS-lab$ make run
[INFO] ysos_kernel - Interrupts Enabled.
[INFO] ysos_kernel - YatSenOS initialized.
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel::interrupt::clock - Tick! @65536
```

可以从 INFO 条数观察到到此时 Tick! 的频率确实减半了，如果需要调整为别的频率，只需更改 TDCR 的值为分频系数的寄存器值即可。

5. 串口输入中断

首先，需要在串口设备初始化 `init` 函数的末尾重新把中断打开

```
// 打开所有中断（IER 寄存器写 1）
unsafe {
    Port::new(base + IER).write(0x01u8);
}
```

同时，为了能够接收到 IO 设备的对应中断，为 IOAPIC 启用对应的 IRQ

```
// FIXED: enable serial irq with IO APIC (use enable_irq)
enable_irq(Irq::Serial0 as u8, 0); // enable IRQ4 for CPU0
```

随后，使用 `crossbeam_queue` crate 实现一个无锁输入缓冲区

1. 处理数据结构的初始化，暴露基本功能

```
// 定义输入缓冲区存储的数据类型（从串口接收的原始字节）
type Key = u8;

// 使用无锁队列作为输入缓冲区，容量 128
lazy_static! {
    static ref INPUT_BUF: ArrayQueue<Key> = ArrayQueue::new(128);
}

/// 将接收到的字符放入输入缓冲区（由中断处理程序调用）
#[inline]
pub fn push_key(key: Key) {
    if INPUT_BUF.push(key).is_err() {
        warn!("Input buffer is full. Dropping key '{:?}'", key);
    }
}

/// 非阻塞地从缓冲区取出一个字符
#[inline]
pub fn try_pop_key() -> Option<Key> {
    INPUT_BUF.pop()
}
```

2. 实现并暴露 `pop_key` 函数

```
/// 阻塞地从缓冲区取出一个字符（循环等待直到有数据）
pub fn pop_key() -> Key {
    loop {
        if let Some(key) = try_pop_key() {
            return key;
        }
    }
}
```

3. 实现并暴露 `get_line` 函数

```
/// 阻塞地读取一行输入，并实时回显，支持退格删除。
/// 遇到换行符 `\n` 或回车符 `\r` 时结束，返回用户输入的字符串（不含换行符）。
pub fn get_line() -> String {
    let mut serial_port = SerialPort::new(0x3F8); // COM1 基地址
    let mut line = String::with_capacity(128);
    loop {
        let ch = pop_key();
        match ch {
            b'\n' | b'\r' => {
                // 换行/回车：打印换行并结束读取
                write!(serial_port, "\n").unwrap();
                break;
            }
            0x08 | 0x7F => { // 退格（Backspace / Delete）
                if !line.is_empty() {
                    line.pop();
                    // 调用串口驱动提供的退格函数，删除屏幕上最后一个字符
                    write!(serial_port, "\x08 \x08").unwrap(); // 退格 + 空格 + 退
格，覆盖最后一个字符
                }
            }
            _ => {
                // 仅处理可打印字符（ASCII 32-126，即空格到波浪号）
                if ch >= 0x20 && ch <= 0x7E {
                    line.push(ch as char);
                    write!(serial_port, "{}", ch as char).unwrap();
                }
                // 忽略其他控制字符（如制表符、铃声等）
            }
        }
    }
    line
}
```

最后，利用上述实现的 `input` 驱动，将接收到的字符放入缓冲区

```
// FIXED: receive character from uart 16550, put it into INPUT_BUFFER
let mut serial_port = uart16550::SerialPort::new(0x3F8); // COM1 基地址
if let Some(byte) = serial_port.receive() {
    push_key(byte);
}
```

6. 用户交互

按照实验文档修改 `src/main.rs` 后，使用 `make run` 运行代码，测试串口输入中断是否正常运行

```
linkzelda@LAPTOP-QFAEHS2I:~/OS-lab$ make run
[INFO] ysos_kernel::memory - Physical Memory    :  95.625 MiB
[INFO] ysos_kernel::memory - Free Usable Memory :  45.012 MiB
[INFO] ysos_kernel::memory - Frame Allocator initialized.
[INFO] ysos_kernel - Interrupts Enabled.
[INFO] ysos_kernel - YatSenOS initialized.
> Hello, World!
You said: Hello, World!
The counter value is 45224
> LinkZelda, 24325005
You said: LinkZelda, 24325005
The counter value is 82020
> exit
[INFO] ysos_kernel - YatSenOS shutting down.
linkzelda@LAPTOP-QFAEHS2I:~/OS-lab$
```

可以看到，内核可以正确根据用户输入进行输出，说明串口输入中断功能的实现无误。

7. 思考题

7.1. 为什么需要在 `clock_handler` 中使用 `without_interrupts` 函数？如果不使用它，可能会发生什么情况？

在 `clock_handler` 中使用 `without_interrupts` 是确保在处理时钟中断进行日志输出的时候不会被其他中断打断。如果不使用它，如果我们的 `COUNTER` 使用的是 `static mut` 而非 `AtomicU64` 类型，则可能会由于操作的非原子性而导致输出日志的计数值和实际不相符；也有可能会和其他模块产生死锁；或者是在输出日志的期间被其他中断打断导致输出的日志不完整。

7.2. 考虑时钟中断进行进程调度的场景，时钟中断的频率应该如何设置？太快或太慢的频率会带来什么问题？请分别回答。

在时钟中断进行进程调度的场景下，通常会将时钟中断的频率设置为 `100~250Hz`，以保证不会由于中断频率过高而带来吞吐量的下降。太快的频率会导致中断处理开销过大，每次处理时钟中断都需要保存和恢复上下文等，大量占用 CPU 和内存等资源，从而使其他的任务所需的执行时间变长，执行等量工作所需的能耗增加；太慢的频率会导致计时的精度下降，时钟中断的周期变长，从而导致某些短时间的任务需要更长的时间调度，使性能降低。

7.3. 在进行 `receive` 操作的时候，为什么无法进行日志输出？如果强行输出日志，会发生什么情况？谈谈你对串口、互斥锁的认识。

因为 `receive` 操作运行在中断上下文，所以它不能调用日志输出这种可能产生阻塞的函数。如果强行输出日志，因为输出日志可能需要获得串口的锁，而此时的锁在 `receive` 操作中，无法释放锁，可能会导致死锁。串口是速度较低的设备，且访问过程中不能被其他进程打断，因此需要用锁来保护，只能同时被一个线程使用。互斥锁指的是当锁被某个线程占有的时候，其他线程会主动让出 CPU，进入等待队列，直到锁被释放后被唤醒。

7.4. 输入缓冲区在什么情况下会满？如果缓冲区满了，用户输入的数据会发生什么情况？

输入缓冲区本质上是一个队列，用户的输入由中断处理程序写入缓冲区的尾部，然后程序会在缓冲区的头部取出数据使用。只有当用户输入的速度高于系统读取的速度的时候，输入缓冲区才可能会满。如果缓冲区满了，可能会丢失用户输入的新数据，也可能会覆盖缓冲区中的旧数据，取决于队列的实现方式（是普通队列还是循环队列）。

7.5. 进行下列尝试，并在报告中保留对应的触发方式及相关代码片段：

- 尝试用你的方式触发 Triple Fault，开启 `intdbg` 对应的选项，在 QEMU 中查看调试信息，分析 Triple Fault 的发生过程。

我们可以通过设置一个非法的 idt 指针，让错误处理的时候由于找不到错误处理程序而产生错误进而产生 Triple Fault，代码如下：

```
pub unsafe fn trigger_triple_fault() {
    use x86_64::instructions::tables::lidt;
    use x86_64::structures::DescriptorTablePointer;
    use x86_64::VirtAddr;

    // 1. 创建一个非法的 IDT 指针
    let ptr = DescriptorTablePointer {
        limit: 0,
        base: VirtAddr::new(0),
    };

    // 2. 加载这个垃圾指针
    lidt(&ptr);

    // 3. 触发一个断点异常，瞬间引发三重错误
    core::arch::asm!("int3");
}
```

产生 Triple Fault 之后，我们可以观察日志中出现了如下四行

```
check_exception old: 0xffffffff new 0xd
check_exception old: 0xd new 0xd
check_exception old: 0x8 new 0xd
Triple fault
```

首先，我们在空的 idt 下调用 `int3` 引发了 GPF（general_protection_fault），对应 `new 0xd`，CPU 尝试取寻找中断处理程序，结果找不到。因而继续引发 `0xd` GPF 异常，此时会触发 `0x8` double_fault，由于 `0x8` 的中断处理程序也找不到，因此再次触发 `0xd` GPF，进而引发 Triple Fault。

- 尝试触发 Double Fault，观察 Double Fault 的发生过程，尝试通过调试器定位 Double Fault 发生时使用的栈是否符合预期。

我们首先通过访问一个非法地址来触发 Page Fault，然后注释掉设置 page_fault_handler 的栈的代码，从而引发 Double Fault，前者的代码如下：

```
fn trigger_page_fault() {
    unsafe {
        let ptr = 0x11451499824 as *mut u8;
        *ptr = 42;
    }
}
```

我们在日志中可以观察到关键的几行，

```
check_exception old: 0xffffffff new 0xe
check_exception old: 0xe new 0xe
v=08 e=0000
```

首先，上述代码触发了一个 `0xe` Page Fault，此时该错误需要把当前的信息压入对应的栈，但由于我们把栈的声明的代码给注释掉了，因此会再次引发一个 `0xe` Page Fault，从而引发一个 `0x8` Double Fault。

我们可以通过输出调试的方法观察一下当前栈的地址，在 `double_fault_handler` 中加入如下代码：

```
let rsp: u64;
unsafe { core::arch::asm!("mov {}, rsp", out(reg) rsp); }
info!("Double Fault RSP: {:#x}", rsp);
```

得到如下的输出，

```
[INFO] ysos_kernel::interrupt::exceptions - Double Fault RSP: 0xffffff0000814fe8
```

对比一下设置给 Double Fault 的栈的地址范围，

```
[INFO] ysos_kernel::memory::gdt - Double Fault IST :
0xffffff0000814100-0xffffff0000815100
```

可知处理 double_fault 使用的栈符合预期。

- 通过访问非法地址触发 Page Fault，观察 Page Fault 的发生过程。分析 Cr2 寄存器的值，并尝试回答为什么 Page Fault 属于可恢复的异常。

我们直接复用上个问题的触发第一层 Page Fault 代码，在日志中可以观察到以下关键的几行，

```
check_exception old: 0xffffffff new 0xe
220: v=0e e=0002 ...
CR2=0000011451499824
```

首先，程序出现了访问非法地址时出现了缺页的错误，引发了 `0xe` Page Fault 错误，错误码为 `0002` 表示是写的时候出现了错误，CR2 中的值为 `0000011451499824`，即我们尝试写入值的地址。

由于 Page Fault 中保存了各个寄存器原本的状态以及 RIP 保存了 PC 寄存器的值，同时 CR2 也给出的访问失败的地址，因此我们可以在处理完这个异常之后直接回到该指令的位置重新执行，也即可恢复。

7.6. 如果在 TSS 中为中断分配的栈空间不足，会发生什么情况? 请分析 CPU 异常的发生过程，并尝试回答什么时候会发生 Triple Fault。

首先，根据上一个思考题，当中断的栈空间不足的时候，会出现 Page Fault，由于此时已经在中断处理程序中了，因此实际上引发的是 Double Fault。如果在 Double Fault 的处理程序中未分配栈空间或者分配的栈空间也不足就会再次导致 Page Fault，此时就会导致 Triple Fault 的发生。

7.7. 在未使用 `set_stack_index` 函数时，中断处理程序的栈可能哪里? 尝试结合 gdb 调试器，找到中断处理程序的栈，并验证你的猜想是否正确。

如果我们不使用 `set_stack_index` 函数，猜想中断处理函数的栈会直接复用内核的栈（因为我们的 TSS 中只设置了特权栈和两个中断栈，没有别的栈可以给它用了）。我们可以选择一个没有使用这个函数的异常，如 Divide Error，用如下代码进行触发：

```
pub unsafe fn trigger_divide_error() {
    unsafe {
        core::arch::asm!("mov eax, 0");
        core::arch::asm!("div eax");
    }
}
```

然后我们直接在对应的 handler 内输出当前的栈指针，对比以下当前栈指针可错误发生时保存的栈指针，

```
[INFO] ysos_kernel::interrupt::exceptions - Divide Error RSP: 0xffffff01001ffe40

stack_pointer: VirtAddr(
        0xffffff01001fff48,
    ),
```

可以发现两者非常相近, 可以推断出是直接复用了内核的栈, 然后保存了中断前进程的上下文，因此我们的猜想正确。

7.8. 结合本实验的 APIC 初始化与中断路径，分析：

- 哪些初始化步骤必须严格按顺序执行?
- 如果你故意交换其中两步，最可能出现的首个可观测故障是什么?

1. SVR 的启用必须先于所有 LVT 配置

在 SVR 的第 8 位 ENABLE_APIC 被置 1 之前，Local APIC 处于“软件禁用”状态。此时对 LVT Timer 的配置可能无法生效。

2. ICR 的高 32 位必须先于低 32 位的写入

在 x86 架构中，向 ICR 的低 32 位（即 0x300）写入会直接触发中断发送动作。如果反过来，中断会发送到上一次存在高 32 位里的目标地址。

7.9. 如果只允许保留一个中断调试手段（日志、寄存器转储、QEMU 调试输出三选一），你会选哪个？请说明理由。

我会选择 QEMU。首先虽然日志输出很直观很方便, 但是对于某些错误, 日志调试可能需要在多出进行日志输出才能得到问题所在, 而且能够日志输出说明串口, 内存分配等都已经正确实现了。对于寄存器转储, 如果遇到了 Triple Fault 的话, 不会产生输出, 而会一直不停地 reboot, 导致不能快速知道发生了什么情况。但 QEMU 调试会输出错误产生的中断处理程序号，对应地错误码和各个寄存器的信息，可以在没有实现串口输出之前用于调试，同时如果出现 Triple Fault, 它会直接在最后标出，便于我们快速找到问题所在。因此我选择 QEMU。

8. 加分项

8.1. 为全部可能的 CPU 异常设置对应的处理程序，使用 panic! 输出异常信息。

已对 `https://wiki.osdev.org/Exceptions` 中展示的所有异常设置了对应的处理程序，并用 `panic!` 输出异常信息, 这里展示部分代码, 其余部分请见 `src/interrupt/exceptions.rs`。

```
// handling GPF
idt.general_protection_fault
    .set_handler_fn(general_protection_fault_handler);

pub extern "x86-interrupt" fn general_protection_fault_handler(
    stack_frame: InterruptStackFrame,
    error_code: u64,
) {
    panic!(
        "EXCEPTION: GENERAL PROTECTION FAULT, ERROR_CODE: 0x{:016x}\n\n{:#?}",
        error_code, stack_frame
    );
}
```

8.2. 你如何定义用于计数的 COUNTER，它能够做到线程安全吗？如果不能，如何修改？

参考实验 0x00 的加分项，为了做到线程安全，我将 `COUNTER` 设置为 `AtomicU64` 类型，并且使用对应的成员函数进行读取时钟和时钟加一操作，保证了时钟操作的原子性。相关代码如下：

```
static COUNTER: AtomicU64 = AtomicU64::new(0);

#[inline]
pub fn read_counter() -> u64 {
    // FIXED: load counter value
    COUNTER.load(core::sync::atomic::Ordering::SeqCst)
}

#[inline]
pub fn inc_counter() -> u64 {
    // FIXED: read counter value and increase it
    COUNTER.fetch_add(1, core::sync::atomic::Ordering::SeqCst) + 1 as u64
}
```

8.3. 操作 APIC 时存在大量比特操作，尝试结合使用 bitflags 和 bit_field 来定义和操作这些寄存器的值，从而获得更好的可读性。

已经重新使用 bitflags 和 bit_field 来重写 `xapic.rs` 的代码，具体请见 `interrupt/apic/xapic.rs`。